

CS & IT ENGINEERING



Algorithms

Analysis of Algorithms

Lecture No.- 1 | 2



By- Aditya Jain sir

Recap of Previous Lecture



Topic

Topic

Loop Complexity



Topics to be Covered



Topic

Topic

Topic

Space Complexity

TC analysis using Recursion Tree Approach.



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on LinkedIn where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : Space Complexity

We define the space used by an algorithm to be the number of function calls (or words) needed to carry out the computation steps required to solve an instance of the problem excluding the space allocated to hold the input. In other word, it is only the work space required by the algorithm.



Topic : Space Complexity

Algo (input)

{

.....

}

Space complexity

↪ Auxiliary space

(Additional space except the input)



Topic : Space Complexity

Eg.3. Linear search

Algo A) Search ($A[n]$, x)

{

for ($i=1$; $i \leq n$; $i++$)

if ($A[i] == x$)

return i ;

}

$$SC = O(1)$$



Topic : Space Complexity

Eg.4. Algo sum A [A[n] , n)

```
{  
    sum = 0;  
    for (i = 1; i ≤ n; i++)  
    {  
        sum = sum + A[i];  
    }  
}
```

SC: O(1)



Topic : Space Complexity

Eg.4. Algo recursive_sum (A, n)

{

 if (n == 1)

 return A[n]

 else

 return (A[n] + recursive_sum (A, n-1));

}

Recursion Stack





Topic : Space Complexity

$$TC \Rightarrow T(n) = T(n-1) + a$$

$$\Rightarrow O(n)$$

Space complexity = $O(n)$

Size = $O(n)$



Auxiliary Space

Recursion stack

Space complexity

The max size of recursion stack at any time during recursion.



Topic : Analysis of Algorithms

#Q. Algo AJ(n)

{

if (n==1)

return;

else

return AJ(n/2);

}

What is the space complexity ?

A

$O(n)$

B

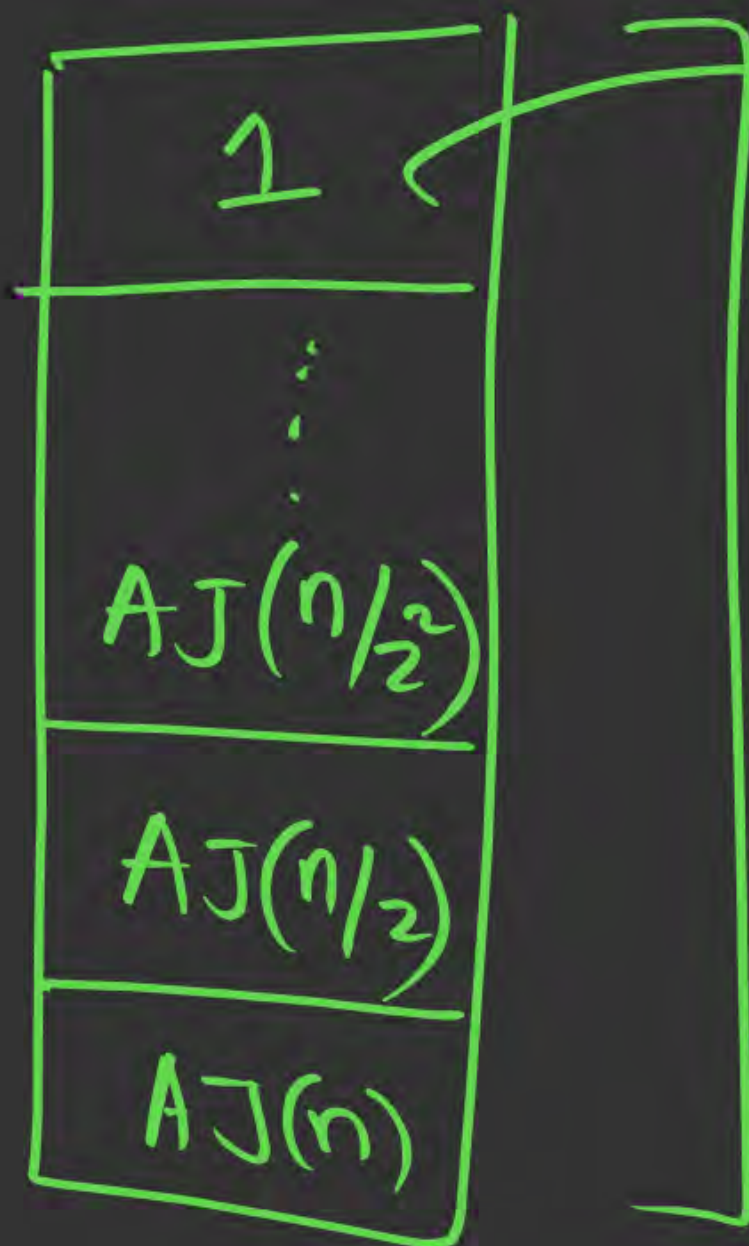
$O(n^2)$

C

$O(n \log n)$

D

$O(\log n)$



$$\frac{n}{2^K} = 1$$

$$2^K = n$$

$$K = \log_2 n$$

$$S(=O(K))$$

$$= \underline{O(\log_2 n)}$$



Topic : Analysis of Algorithms

#Q. Algo AJ(n)

{

int i, j, k = 0;

for (i=n/2; i ≤ n; i = i+1)

{

for (j=2; j ≤ n; j = j*2)

{

k = k + n/2;

{

}

return (k);

}

The return value of the function is ?



nested independent loop

→ $O\left(\frac{n}{2}\right) = O(n)$

→ $O(\log_2 n)$

(B)

A

$O(n)$

~~**B**~~

$O(n^2 \log n)$

C

$O(n \log n)$

D

$O(n^3 \log n)$



Topic : Space Complexity

Time complexity $\Rightarrow O(n \log n)$

$$\text{Overall TC} = O(n \times \log_2 n)$$



Topic : Space Complexity

$$\text{Value of } K \Rightarrow \frac{n \log n}{2} \times \frac{n}{2} \Rightarrow O(n^2 \log n)$$

$$\text{Value of } K = \frac{n}{2} \times \text{runs}$$

$$= \frac{n}{2} \times \left[\frac{n}{2} \times \log_2 n \right]$$

$$= \underline{O(n^2 \log_2 n)}$$



Topic: Miscellaneous



Time Complexity Analysis : (Recurrence Relations)

Approaches	We can find
✓ Back-substitution	Value of Recurrence ✓ TC: notation ✓
Master Method / Theorem → $O(n^C)$	TC: notation ✓
✓ Recurrence / Recursion Tree Approach	TC: notation ✓

$$T(n) = a, n = 1$$

$$T(n) = 2T(n/2) + n, n > 1$$



Topic: Miscellaneous



Solution:

Back- substitution:

$$T(n) = 2T(n/2) + n \quad (1)$$

$$T(n/2) = 2T(n/2^2) + n/2$$

$$T(n) = 2[2T(n/2^2) + n/2] + n$$

$$T(n) = 2^2 T(n/2^2) + 2n \quad (2)$$

$$T(n) = 2^3 T(n/2^3) + 3n \quad (3)$$



Topic: Miscellaneous



General Term

$$T(n) = 2^k T(n/2^k) + k * n \quad (4)$$

for B-C $\boxed{\frac{n}{2^k} = 1} \Rightarrow \underline{\underline{n = 2^k}}$

$$\boxed{k = \log_2^n}$$

$$T(n) = n * T(1) + n * k$$

$$T(n) = n * c + n \log_2^n$$

→ value of recurrence

$$\underline{\underline{T(n) = O(n \log_2 n)}}$$



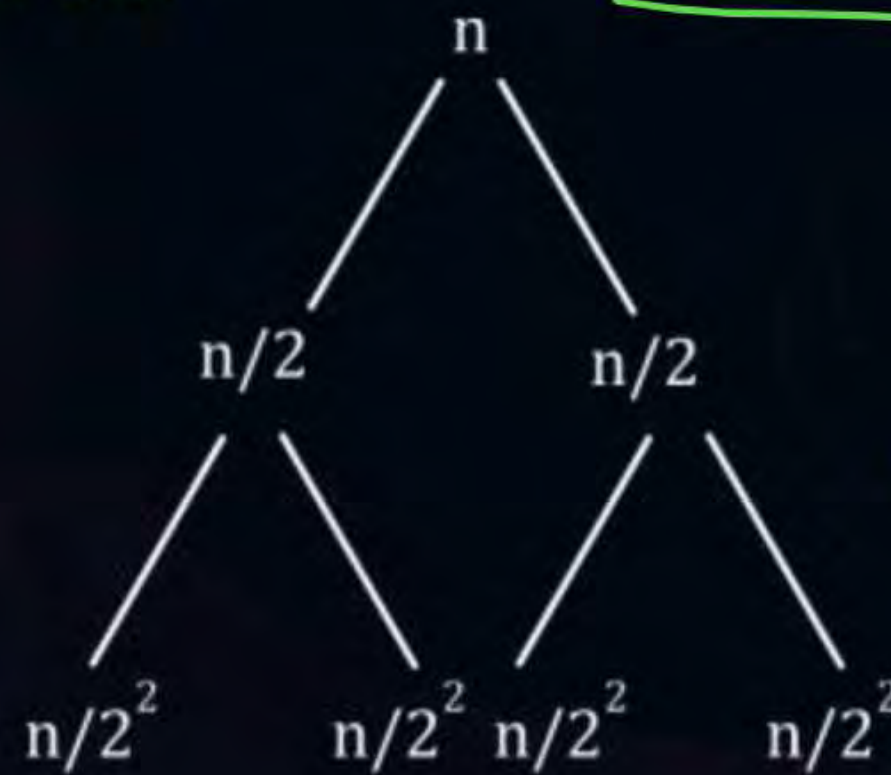
Topic: Miscellaneous



Recurrence tree Approach:

$$T(n) = 2T(n/2) + \textcircled{n}$$

Cost/fee of recursion



total Cost
 n

n

Every level has cost of 'n'.

$\rightarrow n$

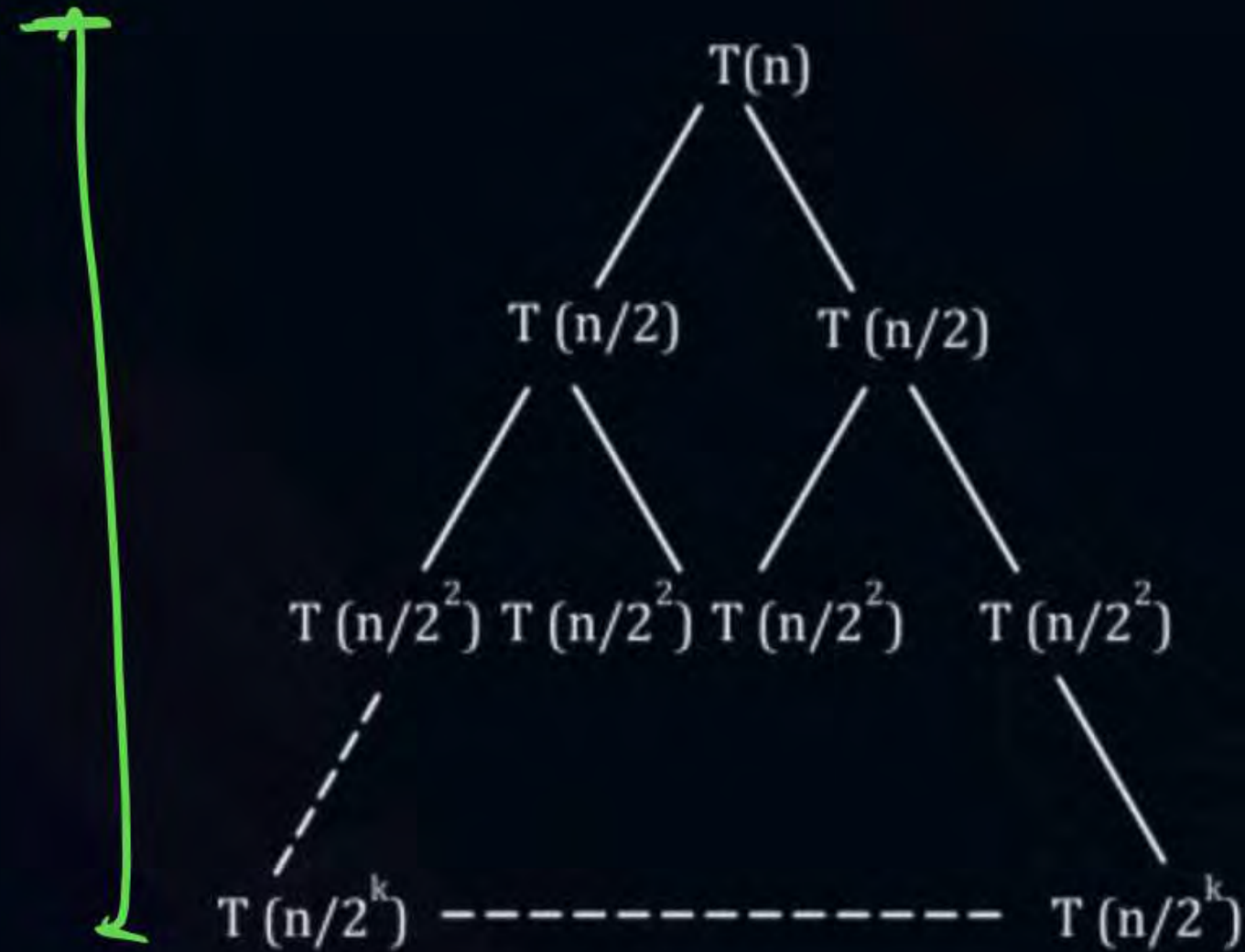


Topic: Miscellaneous



~~K levels~~

$(K+1)$ levels



Assume this is last level (Base condition)



Topic: Miscellaneous



At last level, it should be base condition

$$T(n/2^k)$$

$$\boxed{\frac{n}{2^k} = 1}$$

$$n = 2^k$$

$$\boxed{k = \log_2^n}$$





Topic: Miscellaneous



Level -1 $\rightarrow T(n)$: cost = n ✓

Level -2 $\rightarrow T(n/2)$: cost = n ✓

Level -3 $\rightarrow T(n/2^2)$: cost = n ✓

Level $-(k+1) \rightarrow T(n/2^k)$: cost = n ✓



Topic: Miscellaneous



Total number of levels = $(k+1)$

The cost/ time of every level = n

Hence, total cost/ time = $n * (k+1)$
 $= n * k + n$

We know

$$k = \log_2^n$$

$$= n * \log_2^n + n$$

$$T(n) = \theta(n \log n)$$

$$T(n) = T(n/3) + T(2n/3) + n$$



Topic: Miscellaneous



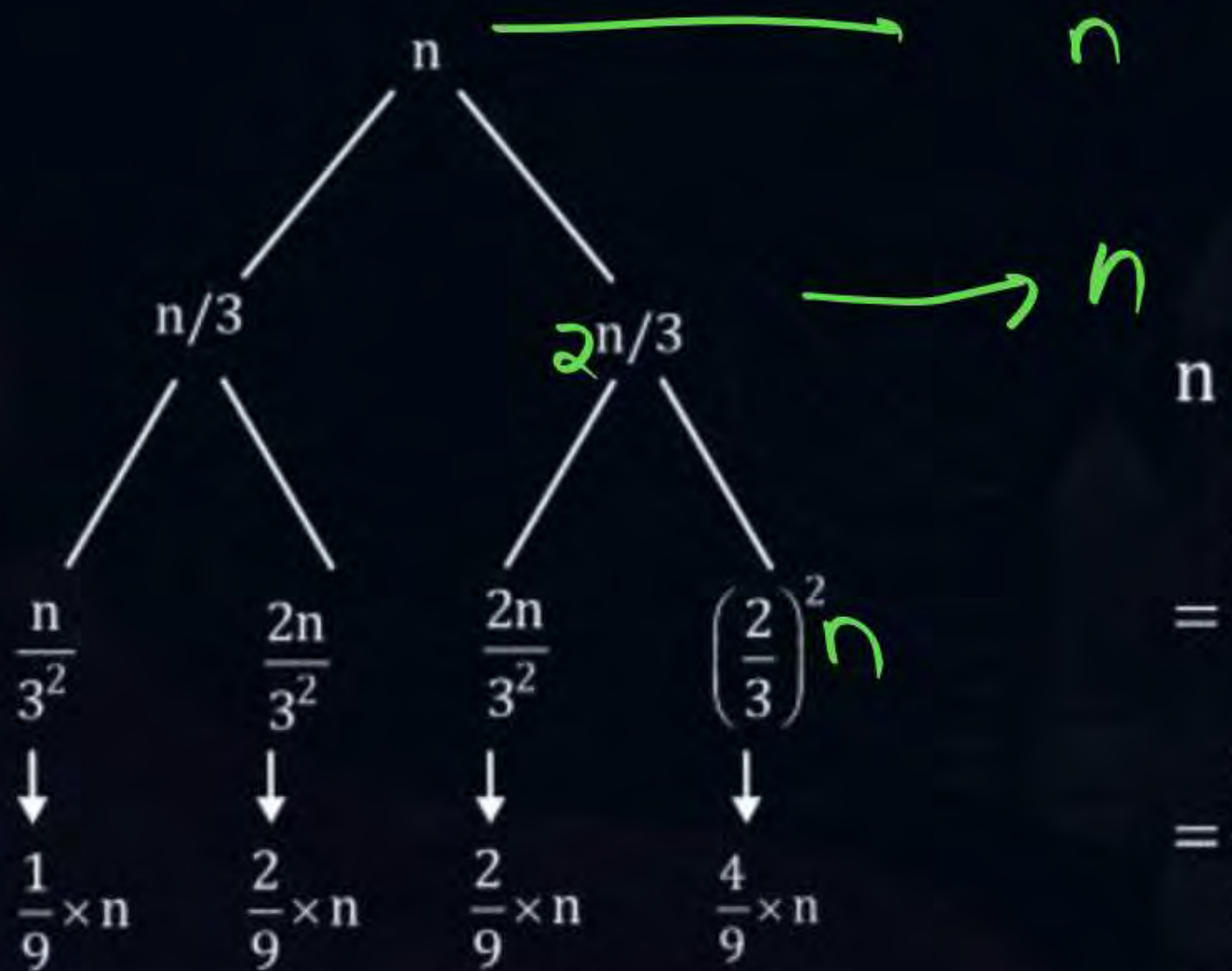
Equation:

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$$

Asymmetric case -1

$$\begin{aligned} n &\rightarrow n/3 \\ n/3 &\rightarrow \frac{n}{3} \times \frac{1}{3} = \frac{n}{9} \end{aligned}$$

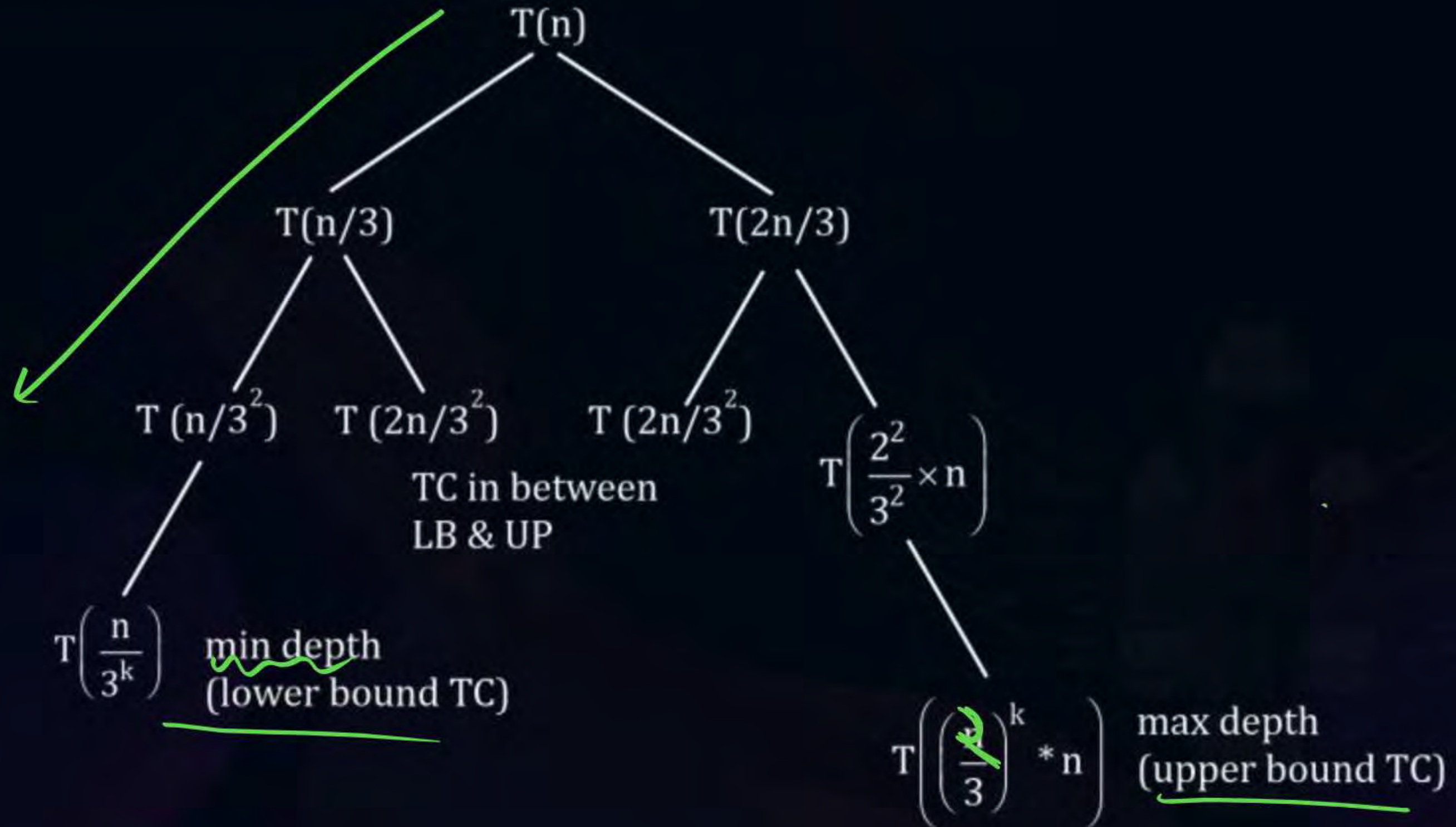
Cost at every level = n



$$\begin{aligned} n &\rightarrow \left(\frac{1}{9} + \frac{2}{9} + \frac{2}{9} + \frac{4}{9} \right) n \\ &= \frac{1+2+2+4}{9} \times n \\ &= \frac{9}{9} \times n \\ &= 1 \times n \end{aligned}$$

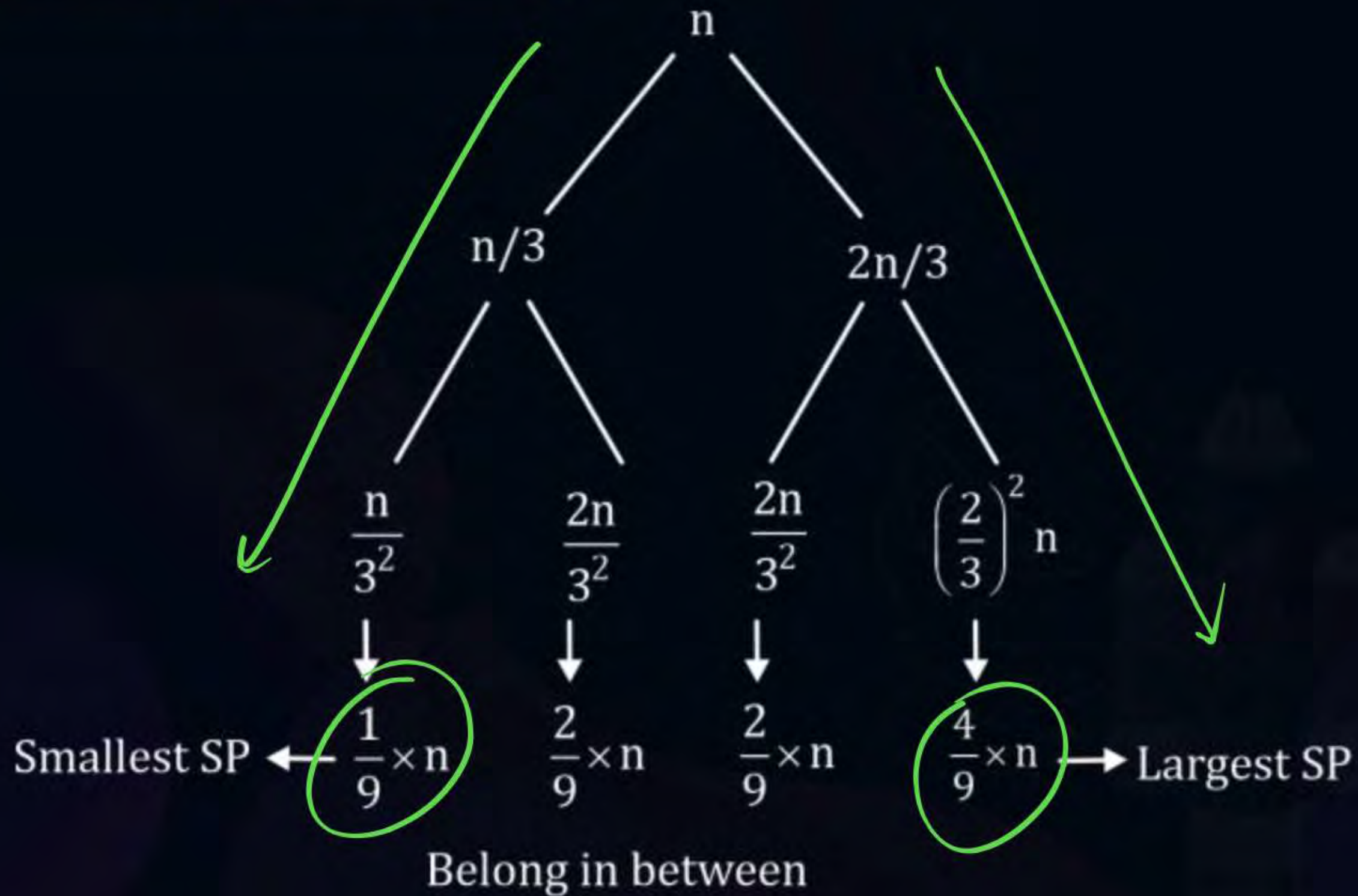


Topic: Miscellaneous



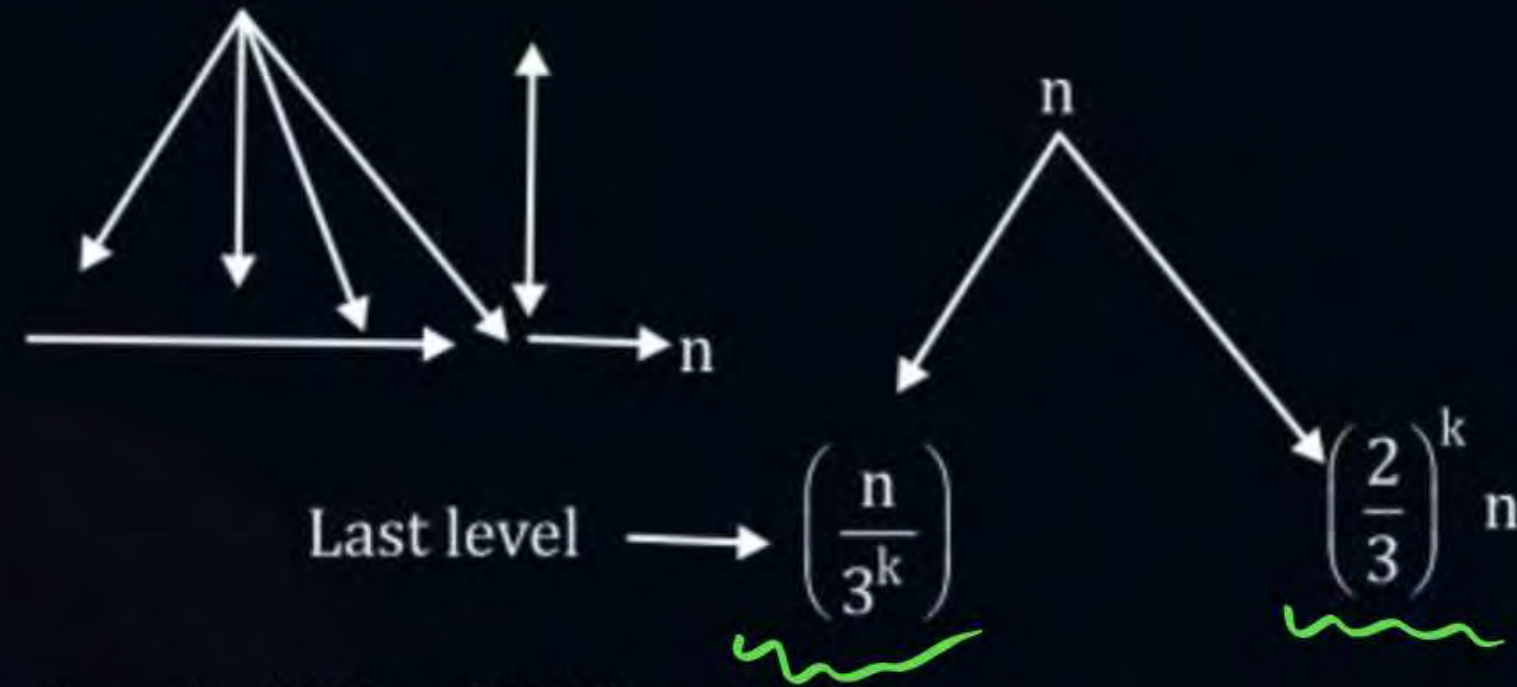


Topic: Miscellaneous





Topic: Miscellaneous



1. For smallest/ min depth (LB of TC)

At least level k , $\frac{n}{3^k} = 1$ (BC)

$$n = 3^k$$

$$k = \log_3 n$$



Topic: Miscellaneous



Assuming there are k levels and cost/time at each level = n

Hence, $T(n) \geq k * n$

$$T(n) \geq \log_3^n * n$$

VB TC

$$\boxed{T(n) \geq n * \log_3^n} \text{ (i)}$$

$$T(n) = \Omega(\log_3^n * n)$$



Topic: Miscellaneous



For largest depth $\rightarrow$ (UB of TC)

Let last level be k

For last level $\left(\frac{2}{3}\right)^k * n = 1$

$$\left(\frac{3}{2}\right)^k = n$$

$$\log n = k * \log \left(\frac{3}{2}\right)$$

$$k = \frac{\log n}{\log \left(\frac{3}{2}\right)} = \log_{3/2}^n$$

$$k = \log_{3/2}^n \quad \checkmark$$

$$\frac{\log^a}{\log^b} = \log_b^a$$



Topic: Miscellaneous



Cost at every level = n

Total cost $\leq k * n$

$$T(n) \leq k * n \overset{=}{\log_{3/2}^n} * n$$

$$T(n) \leq n * \log_{3/2}^n \text{—————(ii)}$$

$$T(n) = O(\log_{3/2}^n)$$



Topic: Miscellaneous



From (i) & (ii)

$$n * \log_3^n \leq T(n) \leq n * \log_{3/2}^n$$

$$T(n) = \Omega(n \log_3 n)$$

$$T(n) = O(n * \log_{3/2}^n)$$

Approximation $\Rightarrow T(n) = \theta(n \log n)$



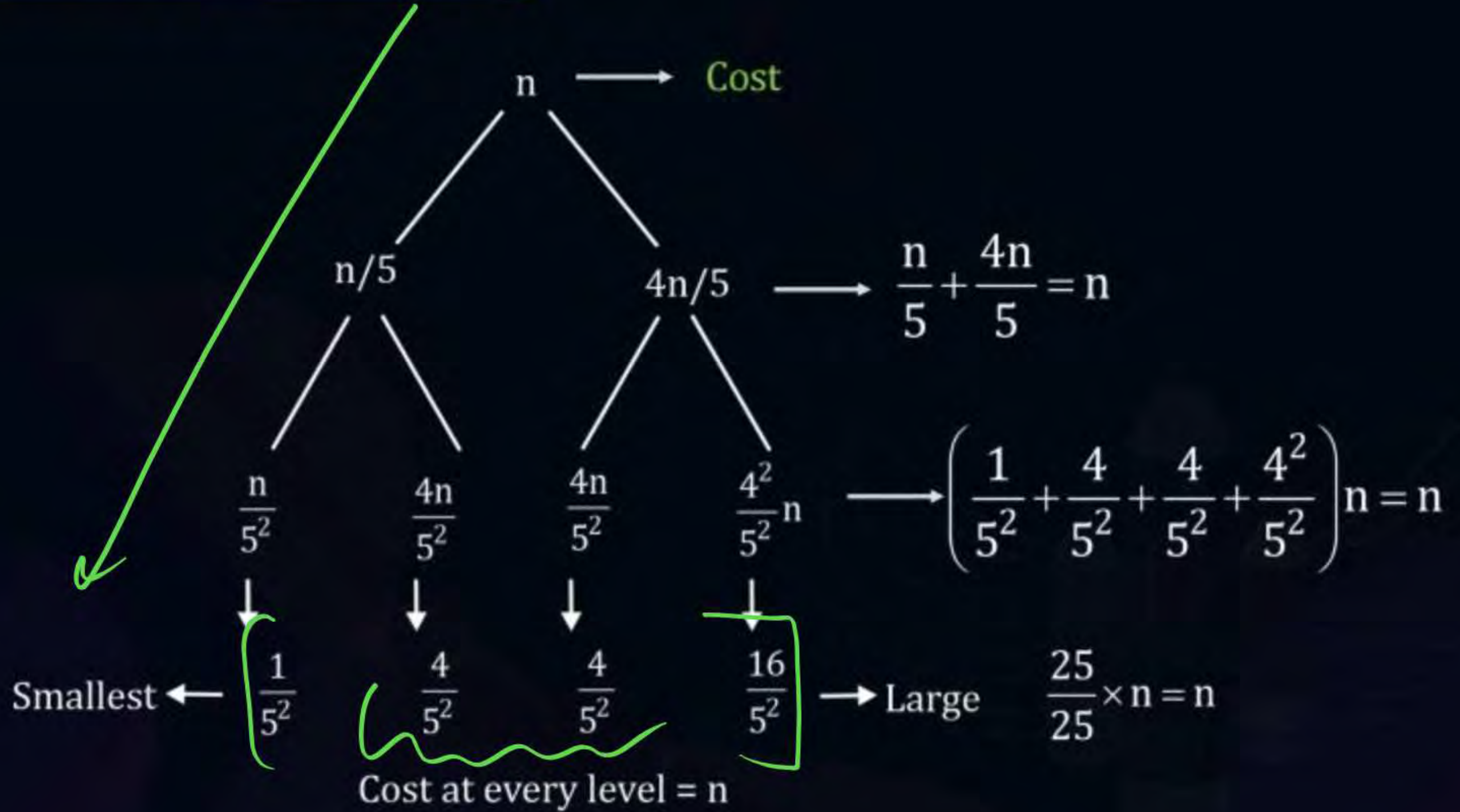
[NAT]



#Q. $T(n) = T(n/5) + T(4n/5) + n$

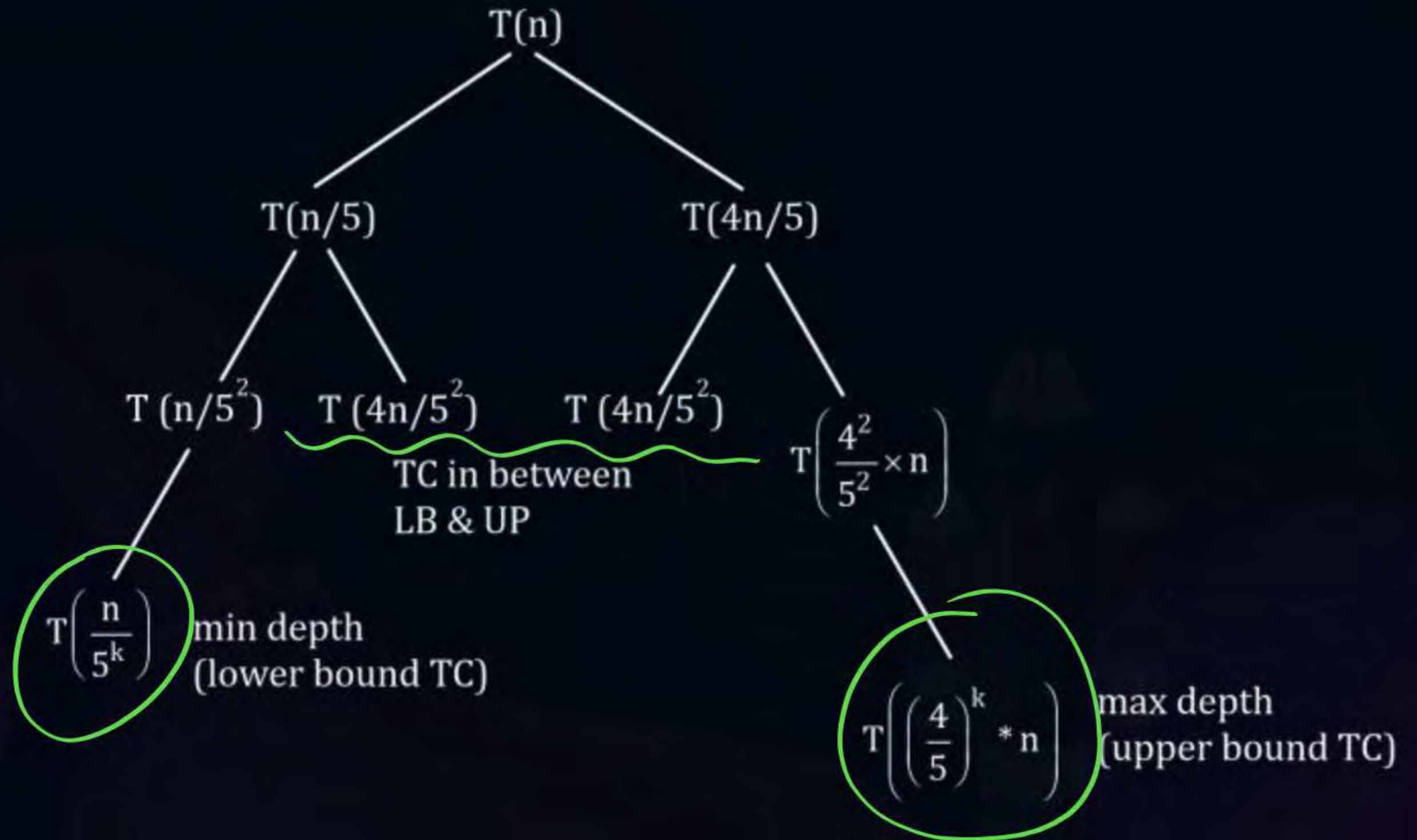


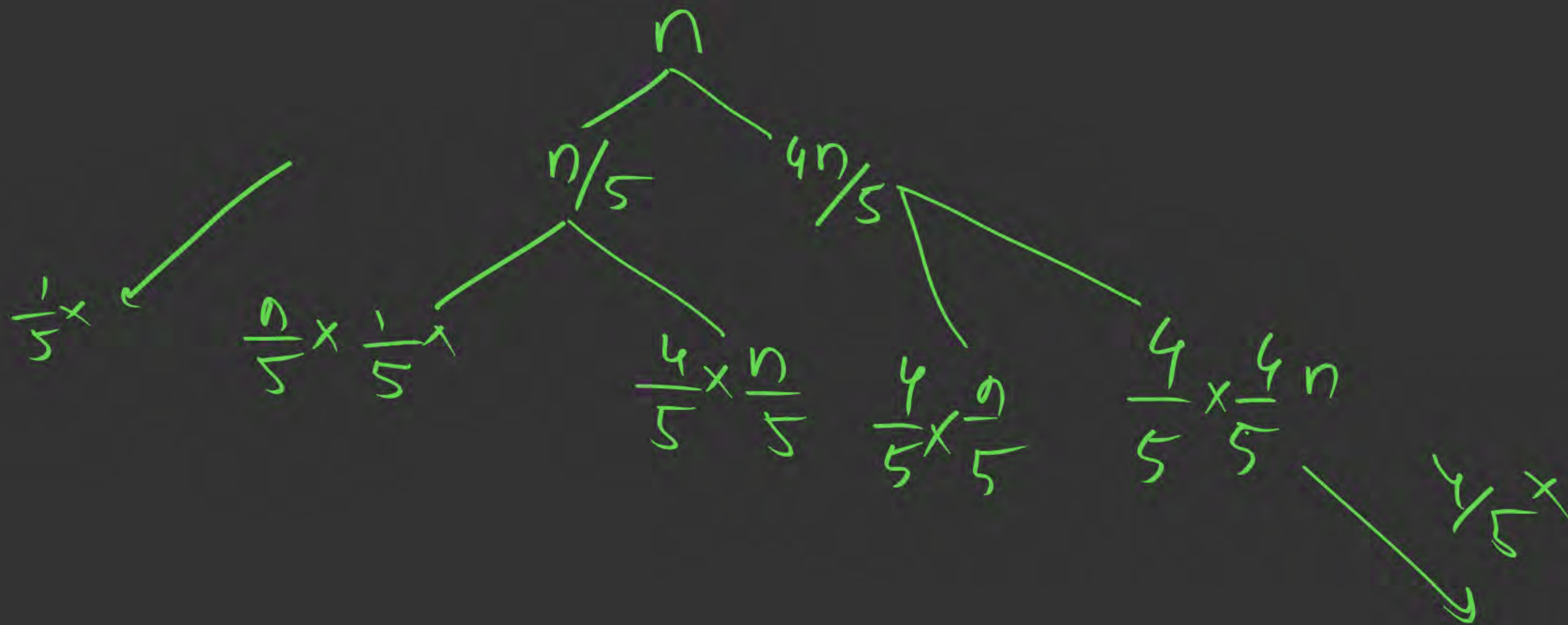
Topic: Miscellaneous





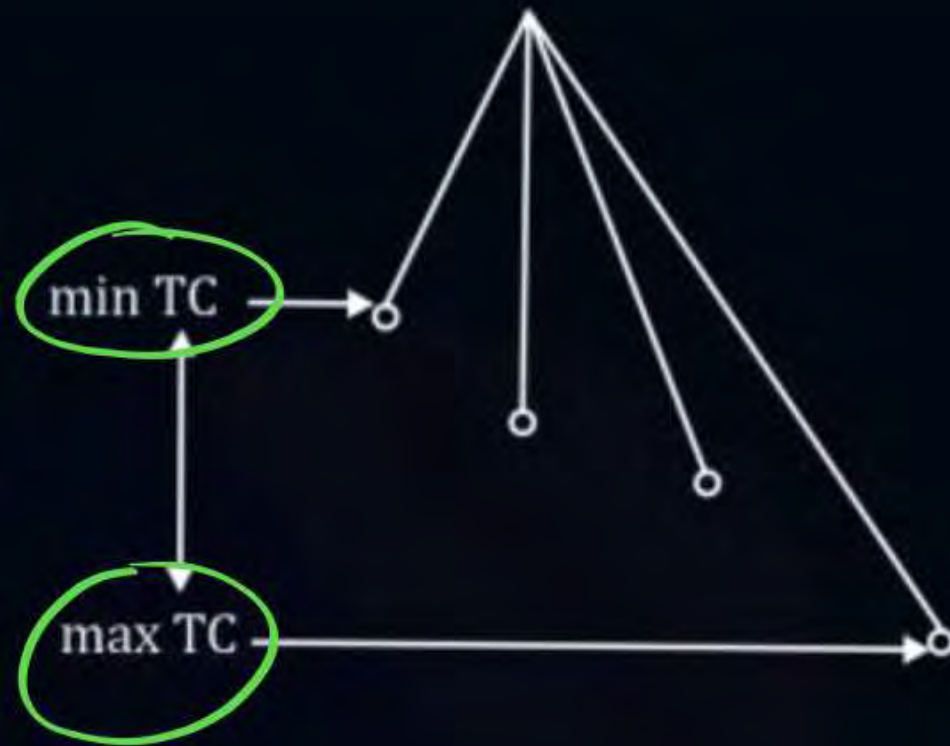
Topic: Miscellaneous







Topic: Miscellaneous





Topic: Miscellaneous



For

Lowest depth (LB) TC

$$\frac{n}{5^k} = 1$$

$$5^k = n$$

$$k = \log_5^n$$

Cost at ~~only~~ ^{any} level = n

$$T(n) \geq n * k$$

$$T(n) \geq n * \log_5^n$$

$$T(n) = \Omega(n \log_5^n)$$





Topic: Miscellaneous



For largest depth:

$$\left(\frac{4}{5}\right)^k n = 1$$

$$\left(\frac{5}{4}\right)^k = n$$

$$k = \log_{5/4}^n$$

$$T(n) \leq n * k$$

$$T(n) \leq n * \log_{5/4}^n$$

$$\underline{T(n) = O(n \log_{5/4}^n)}$$



Topic: Miscellaneous



$$n * \log_5^n \leq T(n) \leq n * \log_{5/4}^n$$

$$T(n) = \Omega(n \log_5^n)$$

$$T(n) = \Theta(n \log_{5/4}^n)$$

Approximation : $T(n) = \theta(n \log n)$

$$T(n) = T(n-1) + T(n-2) + 1$$



Topic: Miscellaneous

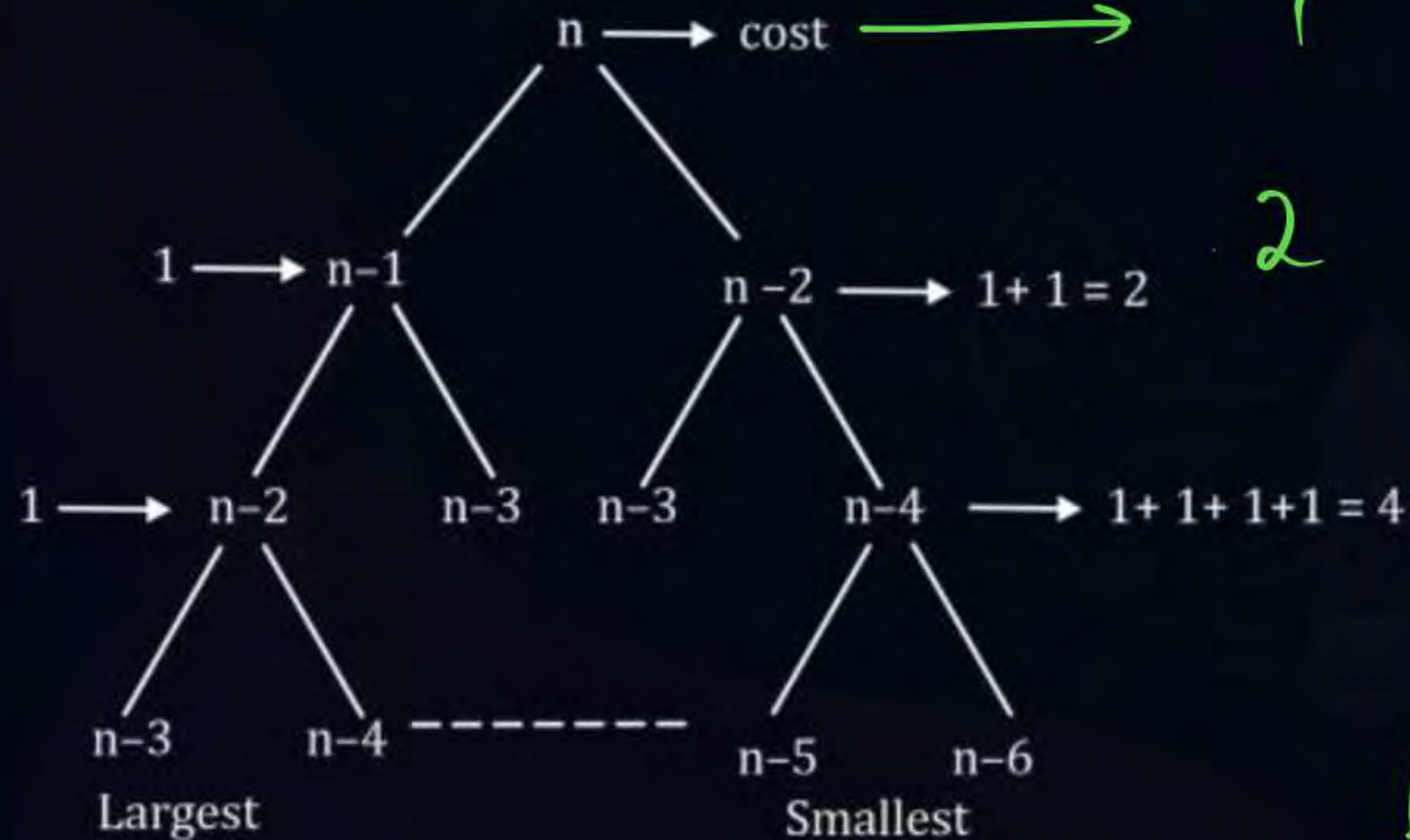
Fibonacci:

~~TC rearrange~~

TC recurrence

$$T(n) = T(n-1) + T(n-2) + 1$$

Cost

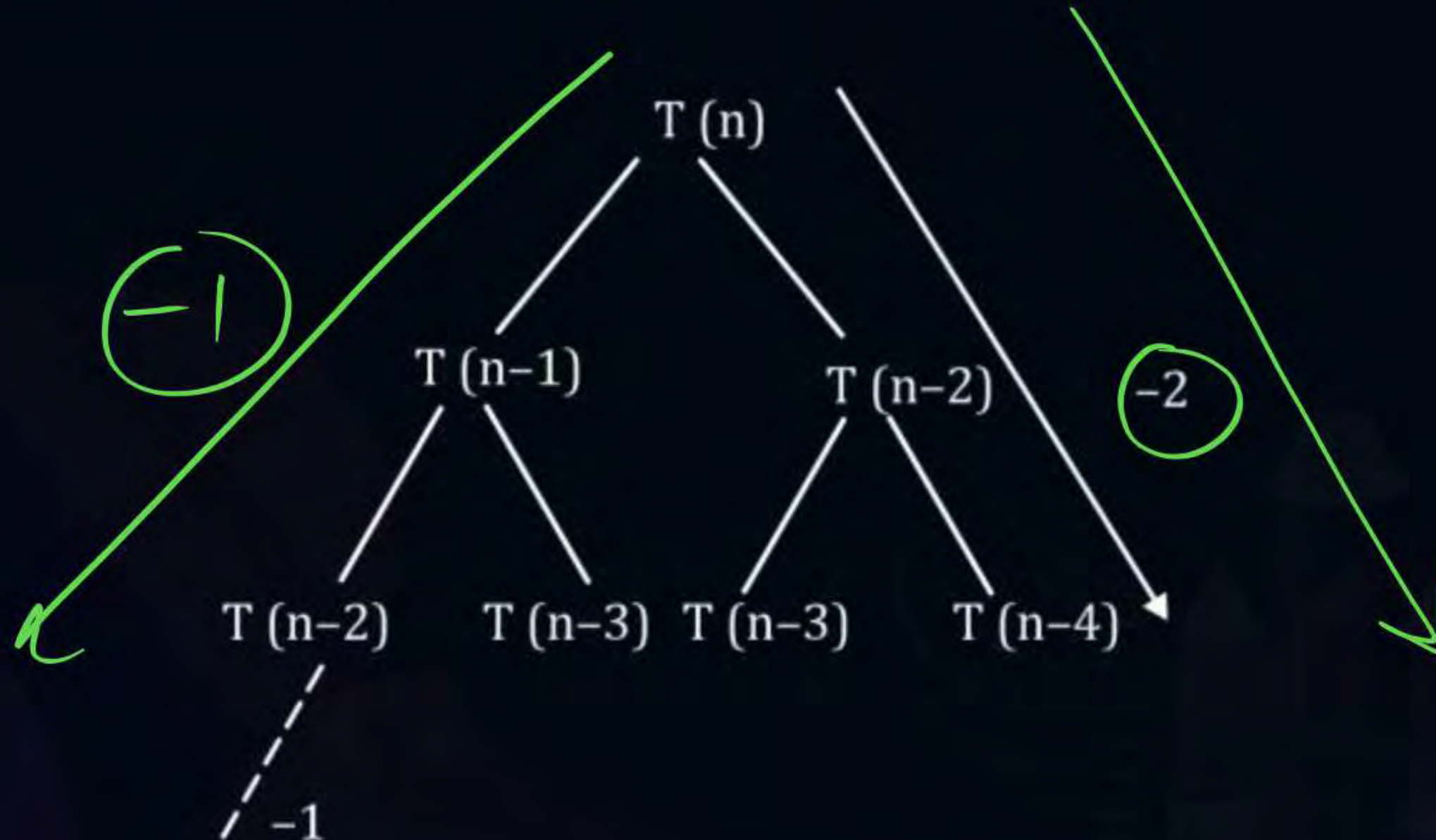


$$2^0$$
$$2^1$$
$$2^2$$
$$\vdots$$
$$2^{K-1}$$

$K \rightarrow 2^{K-1}$



Topic: Miscellaneous





Topic: Miscellaneous



Left –most branch

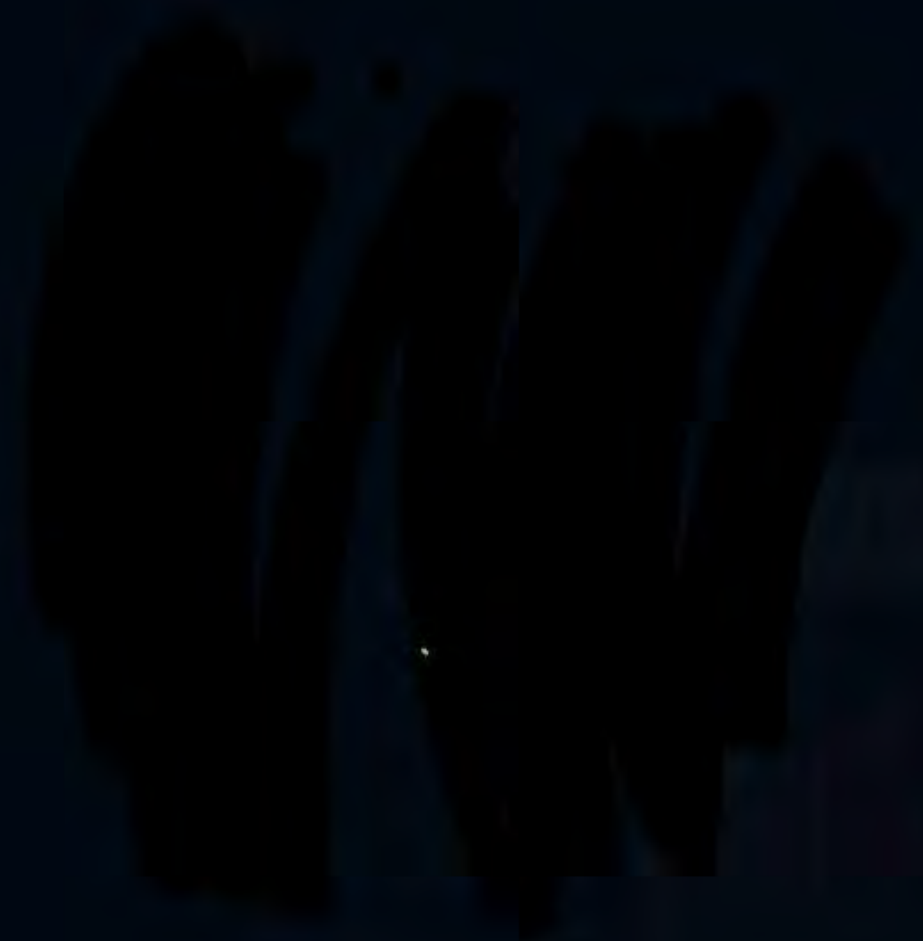
$n-1 \rightarrow n-2 \rightarrow n-3 \dots\dots\dots$

Number of levels $K = n$ (UB TC)

Right –most branch

$n-2 \rightarrow n-4 \rightarrow n-6 \dots\dots\dots$

Number of levels = $n/2$ (LB of TC)





Topic: Miscellaneous



Total Cost till level k

$$= 2^0 + 2^1 + 2^2 + \dots + 2^{k-1} = 2^k - 1$$

1. Left most branch $\rightarrow k=n$

$$T(n) < 2^n - 1$$

$$T(n) = O(2^n)$$



Topic: Miscellaneous



2. Right most branch $\rightarrow k = n/2$

$$T(n) \gg 2^{n/2} - 1$$

$$T(n) = \Omega(2^{n/2})$$



THANK - YOU

